

**INSTITUTO
FEDERAL**

Santa Catarina

Câmpus
São José

Gerência de Memória - Parte II

Sistemas Operacionais

Bernardo Souza Muniz.

2 de Junho de 2026

Engenharia de Telecomunicações - IFSC-SJ

Sumário

1. Hardware de Memória	3
1.1. Descobrimo o Tamanho da Página e Limites do Sistema	3
1.2. Alocação Dinâmica e Páginas Virtuais vs. Físicas	4
1.3. Compilação e Monitoramento com ferramentas ps e /proc	4
1.4. Inspeccionando a Tabela de Páginas com o pmap	6
1.5. Análise de Páginas Globais e Atividade de Swap (/proc/meminfo e vmstat)	7
1.6. Mapeamento de Arquivos em Páginas (mmap)	8
2. Uso de Memória	10
2.1. Código hello.c	10
2.2. Código alocao.c	11
2.3. Código fatorial.c	11
3. MMU (Memory Management Unit)	12

1. Hardware de Memória

1.1. Descobrindo o Tamanho da Página e Limites do Sistema

Para descobrir o tamanho de página, foi executado os seguintes comandos:

```
1 getconf PAGESIZE
2 sysctl kernel.shmall
```

Foi obtido o seguinte resultado:

Figura 1: Elaborada pelo Autor

```
aluno: ~$ getconf PAGESIZE
4096
aluno: ~$ sysctl kernel.shmall
kernel.shmall = 18446744073692774399
aluno: ~$
```

Valor do tamanho de página

- O valor do tamanho de página é de 4096 Bytes (4 KB). Este valor é disponibilizado a um processo sempre quando é solicitado memória para o sistema operacional. Mesmo que um processo precise de somente 1 KB, ainda sim será alocado um tamanho de página de 4 KB para que o programa consiga operar.
- O valor obtido em `kernel.shmall` foi de 18446744073692774399, que representa o número máximo de páginas de memória compartilhada que podem ser alocadas.

Multiplicando valor pelo tamanho de página, temos:

$$18446744073692774399 \times 4096 = 7,555786373 \times 10^{22} \text{ bytes} \quad (1)$$

Convertendo para Gigabytes (GB):

$$75.557.863.737.424.995.964.034 \div 10000000000 = 75.557.863.737.425 \text{ GB} \quad (2)$$

Entretanto, esse valor não corresponde à memória física disponível no computador, sendo apenas um limite teórico para não haver restrição na alocação de memória compartilhada. O limite é imposto pela quantidade de memória física disponíveis no sistema.

Figura 2: Elaborada pelo Autor

```
aluno: ~$ free -h
              total        usada       livre    compart.  buff/cache  disponível
Mem.:          7,6Gi        4,3Gi        219Mi      572Mi        4,0Gi        3,3Gi
Swap:          4,5Gi          0B        4,5Gi
aluno: ~$ █
```

Valor do tamanho de memória física

1.2. Alocação Dinâmica e Páginas Virtuais vs. Físicas

Foi criado o uma arquivo chamado `paginacao.c` e adicionado o seguinte código:

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <string.h>
4  #include <unistd.h>
5  #define DEZESSEIS_MEGA (16 * 1024 * 1024) // 16 MB
6
7  int main() {
8  int pid = getpid();
9  printf("=== PROCESSO INICIADO (PID: %d) ===\n", pid);
10 printf("Passo A: Antes de alocar memoria. Verifique o consumo.\n");
11 printf("Pressione ENTER para alocar 16 MB (mas sem usar)...");
12 getchar();
13 // Alocando 16 Megabytes de memoria virtual
14 char *memoria = (char *)malloc(DEZESSEIS_MEGA);
15 if (memoria == NULL) {
16 perror("Erro ao alocar");
17 return 1;
18 }
19 printf("\nPasso B: 16 MB alocados virtualmente.\n");
20 printf("Pressione ENTER para ESCREVER nas paginas de memoria...");
21 getchar();
22 // Escrevendo em cada pagina (pulando de 4096 em 4096 bytes)
23 // Isso força o Kernel a alocar os quadros fisicos na RAM
24 for (int i = 0; i < DEZESSEIS_MEGA; i += 4096) {
25 memoria[i] = 'X';
26 }
27 printf("\nPasso C: Dados escritos na memoria (Paginas fisicas
associadas).\n");
28 printf("Pressione ENTER para encerrar o programa...");
29 getchar();
30 free(memoria);
31 return 0;
32 }
```

1.3. Compilação e Monitoramento com ferramentas ps e /proc

Para monitorar o funcionamento do programa, foi aberto dois terminais na máquina local. No terminal 1, foi feito a compilação de execução do programa. No terminal 2, foi utilizado comandos utilitários do sistema para coletar os valores da memória virtual (*VSZ/VmSize*) e o tamanho físico presente na RAM (*RSS/VmRSS*).

Ao executar o programa, foi retornado o valor de PID 15987. No terminal 2, foi executado os seguintes comandos para monitoramento da memeória:

```
1 ps -o pid,vsz,rss,comm -p 15987
2 cat /proc/[PID]/status | grep -E "VmSize|VmRSS"
```

Figura 3: Elaborada pelo Autor

The image shows two terminal windows side-by-side. The left window shows the execution of a program named 'paginacao.c'. The user runs './paginacao' and the program outputs '=== PROCESSO INICIADO (PID: 15987) ===' and 'Passo A: Antes de alocar memoria. Verifique o consumo. Pressione ENTER para alocar 16 MB (mas sem usar)...'. The right window shows the user running 'ps -o pid,vsz,rss,comm -p 15987' which outputs a table with columns PID, VSZ, RSS, and COMMAND. The output shows PID 15987, VSZ 2468, and RSS 936 for the command 'paginacao'. Then, the user runs 'cat /proc/15987/status | grep -E "VmSize|VmRSS"' which outputs 'VmSize: 2468 kB' and 'VmRSS: 936 kB'.

Execução do programa `paginacao.c` e monitoramento da memória virtual do processo

Ao avançar a execução do código com o botão de ENTER no terminal 1 e monitorar a alocação de memória do processo no terminal 2, foi obtido o seguinte resultado:

Figura 4: Elaborada pelo Autor

The image shows two terminal windows side-by-side. The left window shows the execution of a program named 'paginacao.c'. The user runs './paginacao' and the program outputs '=== PROCESSO INICIADO (PID: 15987) ===' and 'Passo A: Antes de alocar memoria. Verifique o consumo. Pressione ENTER para alocar 16 MB (mas sem usar)...'. Then, the user presses ENTER and the program outputs 'Passo B: 16 MB alocados virtualmente. Pressione ENTER para ESCREVER nas paginas de memoria...'. Then, the user presses ENTER and the program outputs 'Passo C: Dados escritos na memoria (Paginas fisicas associadas). Pressione ENTER para encerrar o programa...'. The right window shows the user running 'cat /proc/15987/status | grep -E "VmSize|VmRSS"' three times. The first output shows 'VmSize: 2468 kB' and 'VmRSS: 936 kB'. The second output shows 'VmSize: 18856 kB' and 'VmRSS: 936 kB'. The third output shows 'VmSize: 18856 kB' and 'VmRSS: 17772 kB'.

Execução dos passos A, B e C do programa `paginacao.c`

Para os valores do tamanho da memória virtual e o tamanho físico alocado na RAM, foi preenchido a seguinte tabela:

Tabela 1: Elaborada pelo Autor

Momento do Programa	VSZ / VmSize (KB)	RSS / VmRSS (KB)
Passo A (Início)	2468	936
Passo B (malloc feito)	18856	936
Passo C (Escrita feita)	18856	17772

Tabela de resultados de compilação

- Quando o código executou o comando `malloc`, a memória virtual (*VSZ/VmSize*) aumentou de 2468 KB para 18856 KB. A diferença entre os dois valores corresponde aos 16 MB que foram alocados.
- A memória física (*RSS/VmRSS*) permaneceu constante com o valor de 936 KB.
- Isso acontece devido ao fato de que o sistema operacional Linux utiliza *Lazy Allocation* para alocação de memória. Quando o programa solicita memória usando `malloc`, o sistema reserva as páginas no espaço virtual de endereçamento do processo, mas adia a associação de frames físicos na RAM até que o programa tente escrever dados nelas. Por isso a memória virtual cresce, mas a memória física fica sem alterações.

- Após o passo C, a memória física (*RSS/ VmRSS*) foi alterada para 17772 KB, pois o programa escreveu dados na RAM física.
- O pulo de 4096 bytes no laço for está ligado ao tamanho de página que é alocado para cada processo quando é solicitado memória ao sistema operacional (4 KB).

1.4. Inspeccionando a Tabela de Páginas com o pmap

Foi mantido a execução do programa anterior parado exatamente no passo C. Foi aberto um terminal secundário e executado o utilitário de mapeamento de processos (*pmap*) com base no PID 15987:

Figura 5: Elaborada pelo Autor

```
aluno: ~$ pmap -x 15987
15987:  ./paginacao
```

Endereço	Kbytes	RSS	Suja	Modo	Mapeamento
0000557ec2870000	4	4	0	r----	paginacao
0000557ec2871000	4	4	0	r-x--	paginacao
0000557ec2872000	4	4	0	r----	paginacao
0000557ec2873000	4	4	4	r----	paginacao
0000557ec2874000	4	4	4	rw---	paginacao
0000557ec8fff000	132	4	4	rw---	[anôn]
00007f9730d9b000	16400	16392	16392	rw---	[anôn]
00007f9731d9f000	152	152	0	r----	libc.so.6
00007f9731dc5000	1368	812	0	r-x--	libc.so.6
00007f9731f1b000	332	128	0	r----	libc.so.6
00007f9731f6e000	16	16	16	r----	libc.so.6
00007f9731f72000	8	8	8	rw---	libc.so.6
00007f9731f74000	52	20	20	rw---	[anôn]
00007f9731fa7000	8	4	4	rw---	[anôn]
00007f9731fa9000	4	4	0	r----	ld-linux-x86-64.so.2
00007f9731faa000	152	152	0	r-x--	ld-linux-x86-64.so.2
00007f9731fd0000	40	40	0	r----	ld-linux-x86-64.so.2
00007f9731fda000	8	8	8	r----	ld-linux-x86-64.so.2
00007f9731fdc000	8	8	8	rw---	ld-linux-x86-64.so.2
00007fff42164000	132	12	12	rw---	[pilha]
00007fff42194000	16	0	0	r----	[anôn]
00007fff42198000	8	4	0	r-x--	[anôn]

kB total	18856	17784	16480		

```
aluno: ~$
```

Monitoramento de processos usando pmap

- É possível verificar na imagem acima a linha correspondente à alocação que foi efetuada.
- Os valores para essa alocação referente as colunas Kbytes, RSS e Dirty são respectivamente 16400, 16392 e 16392.
- A coluna Suja (Dirty), indica a quantidade de memória virtual que foi escrita pelo processo na RAM dentro do laço for. Convertendo o valor observado, é possível ver que 16 KB foram escritos dentro da memória RAM.

1.5. Análise de Páginas Globais e Atividade de Swap (/proc/meminfo e vmstat)

Foi executado o seguinte comando no terminal para inspecionar os contadores de páginas de cache de disco mantidos pelo Kernel:

```
1 cat /proc/meminfo | grep -E "Cached|Buffers"
```

Foi obtido o seguinte retorno com a execução do comando:

Figura 6: Elaborada pelo Autor

```
aluno: ~$ cat /proc/meminfo | grep -E "Cached|Buffers"
Buffers:                158472 kB
Cached:                 2987288 kB
SwapCached:              0 kB
aluno: ~$ █
```

Inspeção dos contadores de paginas de cache de disco

Foi aberto várias abas de navegador de internet e utilizado o utilitário vmstat para atualizar as estatísticas de desempenho a cada 2 segundos do sistema operacional:

Figura 7: Elaborada pelo Autor

```
procs -----memória----- --swap-- ----e/s---- -sistema- -----cpu-----
r b  swpd  livre  buff  cache  si  so    bi  bo    in  cs  us  sy  id  wa  st
2 0    0 1934824 158496 3097832 0 0    0 0    0 389 1162 2 0 98 0 0
3 0    0 1934824 158496 3097832 0 0    0 0    0 744 2176 5 1 94 0 0
1 0    0 1934824 158496 3097832 0 0    0 0 1350 6033 14 4 82 0 0
1 0    0 1934824 158496 3097832 0 0    0 0 1600 5785 19 4 77 0 0
1 0    0 1934824 158496 3097832 0 0    0 22 606 1566 5 1 94 0 0
6 0    0 1934824 158496 3097832 0 0    0 0 654 1899 6 1 93 0 0
2 0    0 1934824 158496 3097832 0 0    18 782 6392 17224 38 7 55 0 0
3 0    0 1934824 158496 3097832 0 0 240 1528 3474 12955 34 4 60 2 0
2 0    0 1934824 158496 3097832 0 0    86 10834 6720 22028 37 9 51 3 0
2 0    0 1934824 158496 3097832 0 0    654 1012 2590 11712 26 4 70 0 0
2 0    0 1934824 158496 3097832 0 0    0 574 1758 6393 21 4 75 1 0
2 0    0 1934824 158496 3097832 0 0    106 1408 2386 8820 26 3 71 0 0
3 0    0 1934824 158496 3097832 0 0 1494 442 4691 18229 42 4 53 0 0
3 0    0 1934824 158496 3097832 0 0    64 802 4857 19245 47 7 45 0 0
5 0    0 1934824 158496 3097832 0 0    0 1494 3889 20090 42 5 52 1 0
2 0    0 1934824 158496 3097832 0 0 760 354 4722 15421 36 8 56 0 0
4 0    0 1934824 158496 3097832 0 0    78 718 2984 16485 30 5 65 0 0
2 0    0 1934824 158496 3097832 0 0    0 1082 6350 17004 35 7 56 1 0
1 0    0 1934824 158496 3097832 0 0    0 180 2539 11329 18 3 80 0 0
2 0    0 1934824 158496 3097832 0 0    0 1182 4772 19575 24 6 70 0 0
5 0    0 1934824 158496 3097832 0 0    32 402 2775 13751 19 3 78 0 0
procs -----memória----- --swap-- ----e/s---- -sistema- -----cpu-----
r b  swpd  livre  buff  cache  si  so    bi  bo    in  cs  us  sy  id  wa  st
1 0    0 1934824 158496 3097832 0 0    54 7788 4418 21367 25 4 70 1 0
2 0    0 1934824 158496 3097832 0 0    0 178 2486 8165 19 3 78 0 0
3 0    0 1934824 158496 3097832 0 0    64 770 2924 12555 32 5 62 1 0
3 0    0 1934824 158496 3097832 0 0 310 3224 2894 10986 43 5 52 0 0
1 0    0 1934824 158496 3097832 0 0    0 214 2252 9434 33 3 63 0 0
```

Monitoramento das estatísticas de desempenho

- As colunas de si (*swap in*) e so (*swap out*), indicam, respectivamente, a quantidade de memória que está sendo trazida do disco de volta para a RAM física e a quantidade de memória que está sendo salva da RAM física para o disco.

Executando apenas o comando `cat /proc/meminfo`, é possível encontrar linhas relacionadas ao termo chamado HugePages.

Figura 8: Elaborada pelo Autor

```
aluno: ~$ cat /proc/meminfo | grep HugePages
AnonHugePages:      1095680 kB
ShmemHugePages:     417792 kB
FileHugePages:       0 kB
HugePages_Total:     0
HugePages_Free:      0
HugePages_Rsvd:      0
HugePages_Surp:      0
aluno: ~$
```

Retorno do comando `cat /proc/meminfo`

HugePages se trata de um recurso de gerenciamento de memória que permite a utilização de paginação com tamanhos maiores que 4 KB. Normalmente as *HugePages* medem 2 MB ou 1 GB e são destinadas a programas que necessitam de uma alocação de memória muito alta. Para não ter que gerenciar muitas de páginas com um tamanho de 4 KB, as *HugePages* permitem a mesma disponibilidade de armazenamento de dados mas com uma gerência menor de quantidade de páginas.

1.6. Mapeamento de Arquivos em Páginas (mmap)

Foi inicialmente criado um arquivo de texto para testes usando o terminal:

```
1 echo "Arquitetura de Computadores e Sistemas Operacionais" > texto.txt
```

Posteriormente foi criado um arquivo chamado `mapeado.c`:

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <fcntl.h>
4 #include <sys/mman.h>
5 #include <sys/stat.h>
6 #include <unistd.h>
7 int main() {
8     int fd = open("texto.txt", O_RDWR);
9     if (fd < 0) {
10         perror("Erro ao abrir arquivo");
11         return 1;
12     }
```



```

12 }
13 struct stat sb;
14 fstat(fd, &sb);
15 // Mapeando o arquivo diretamente em uma pagina de memoria virtual
16 char *arquivo_em_memoria = mmap(NULL, sb.st_size, PROT_READ | PROT_WRITE,
17 MAP_SHARED, fd, 0);
18 if (arquivo_em_memoria == MAP_FAILED) {
19 perror("Erro no mmap");
20 return 1;
21 }
22 // O arquivo agora pode ser lido como se fosse um vetor na memoria
23 printf("Conteudo lido da pagina de memoria: %s", arquivo_em_memoria);
24 // Modificando a memoria diretamente. A MMU e o Kernel sincronizarao com
25 // o disco.
26 arquivo_em_memoria[0] = 'O';
27 printf("Modificacao feita na memoria. Pressione ENTER para
28 finalizar...");
29 getchar();
30 munmap(arquivo_em_memoria, sb.st_size);
31 close(fd);
32 return 0;
33 }

```

Após compilar e executar o programa, foi verificado o conteúdo final do arquivo de texto criado anteriormente:

Figura 9: Elaborada pelo Autor

```

aluno: ~$ ./mapeado
Conteudo lido da pagina de memoria: Arquitetura de Computadores e Sistemas Operacionais
Modificacao feita na memoria. Pressione ENTER para finalizar...
aluno: ~$ cat texto.txt
Orquitetura de Computadores e Sistemas Operacionais

```

Execução do código mapeado.c

- É possível verificar que a letra “A” da palavra “*arquitetura*” foi substituída pela letra “O”. O mecanismo que permite mapear arquivos em disco diretamente no espaço de endereçamento virtual de um processo, se chama *mmap*.
- A função *mmap()*, presente na linha 16, permite criar um mapeamento entre o arquivo que está em disco e as páginas da memória virtual do processo. Inicialmente o arquivo é associado a uma região de endereçamento virtual do processo pertencente ao código em execução.
- Quando o código acessa essa região, a MMU (*Memory Management Unit*) carrega as páginas correspondentes ao arquivo para a memória RAM. O ponteiro *arquivo_em_memoria* aponta para as páginas que foram carregadas pela MMU e qualquer alteração feita nessa memória marca a página com uma memória suja (*dirty*).
- Por fim, quando as chamadas *munmap()* e *close()* são feitas, as páginas que foram modificadas são gravadas no disco.

2. Uso de Memória

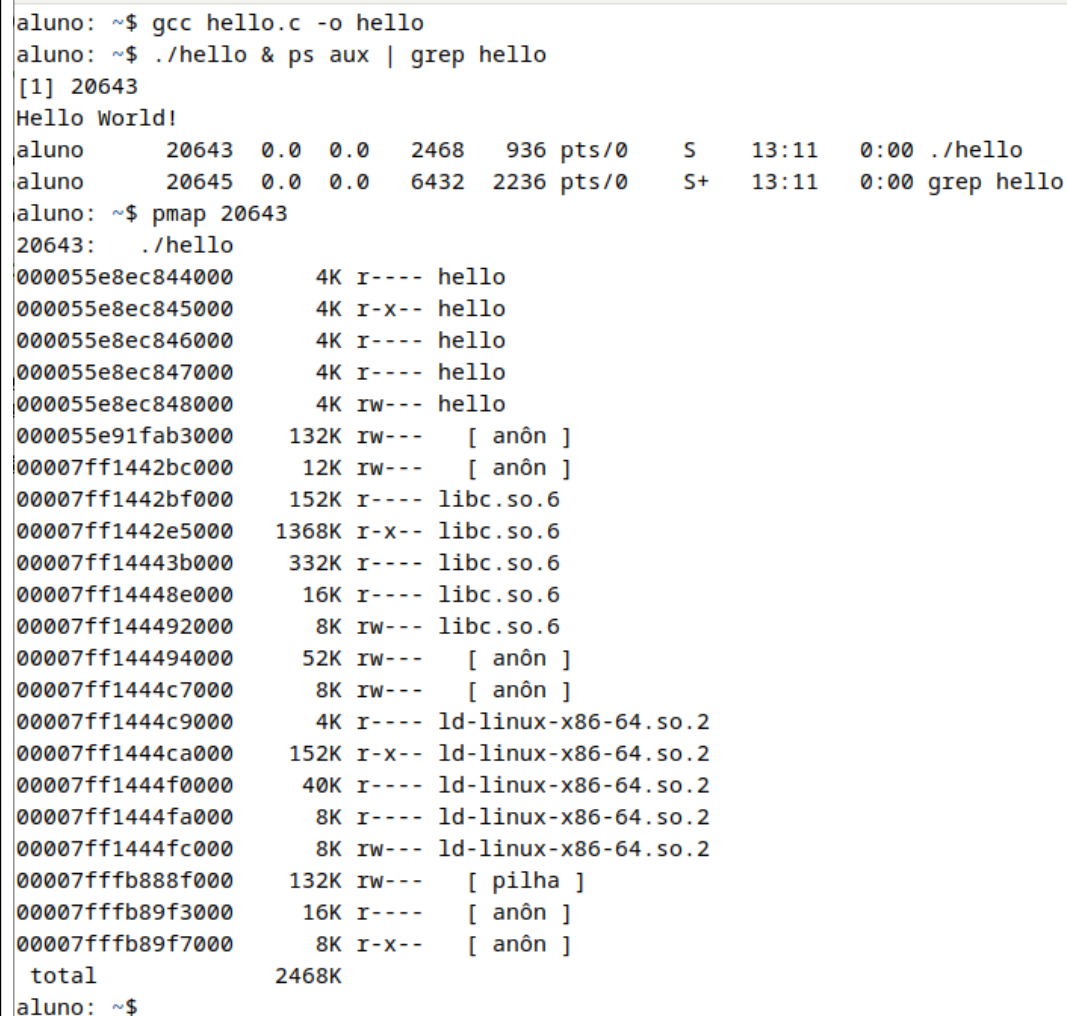
2.1. Código hello.c

Foi criado o seguinte código:

```
1  #include <stdio.h>
2  #include <unistd.h>
3  int global_init = 42; // DATA
4  int global_uninit; // BSS
5  int main() {
6  printf("Hello World!\n");
7  sleep(600);
8  }
```

Para monitorar o uso de memória, foi mapeado utilizando o comando pmap.

Figura 10: Elaborada pelo Autor



```
aluno: ~$ gcc hello.c -o hello
aluno: ~$ ./hello & ps aux | grep hello
[1] 20643
Hello World!
aluno      20643  0.0  0.0   2468   936 pts/0    S   13:11   0:00 ./hello
aluno      20645  0.0  0.0   6432  2236 pts/0    S+  13:11   0:00 grep hello
aluno: ~$ pmap 20643
20643:  ./hello
000055e8ec844000      4K r---- hello
000055e8ec845000      4K r-x-- hello
000055e8ec846000      4K r---- hello
000055e8ec847000      4K r---- hello
000055e8ec848000      4K rw--- hello
000055e91fab3000    132K rw--- [ anôn ]
00007ff1442bc000     12K rw--- [ anôn ]
00007ff1442bf000    152K r---- libc.so.6
00007ff1442e5000   1368K r-x-- libc.so.6
00007ff14443b000    332K r---- libc.so.6
00007ff14448e000     16K r---- libc.so.6
00007ff144492000      8K rw--- libc.so.6
00007ff144494000    52K rw--- [ anôn ]
00007ff1444c7000      8K rw--- [ anôn ]
00007ff1444c9000      4K r---- ld-linux-x86-64.so.2
00007ff1444ca000   152K r-x-- ld-linux-x86-64.so.2
00007ff1444f0000     40K r---- ld-linux-x86-64.so.2
00007ff1444fa000      8K r---- ld-linux-x86-64.so.2
00007ff1444fc000      8K rw--- ld-linux-x86-64.so.2
00007fffb888f000   132K rw--- [ pilha ]
00007fffb89f3000     16K r---- [ anôn ]
00007fffb89f7000      8K r-x-- [ anôn ]
total                2468K
aluno: ~$
```

Execução e monitoramento do código hello.c

2.2. Código alocaao.c

Foi criado o seguinte código para exemplificar a alocação estática, automática e dinâmica:

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 int global = 10; // Estática
4 void funcao_auto(int n) {
5     int local = n; // Automática
6     printf("Local = %d\n", local);
7 }
8 int main() {
9     funcao_auto(5);
10    int *ptr = malloc(sizeof(int)); // Dinâmica
11    *ptr = 99;
12    printf("Dinamica = %d\n", *ptr);
13    free(ptr);
14    return 0;
15 }
```

Foi realizado a compilação e execução do código:

Figura 11: Elaborada pelo Autor

The image shows a terminal window titled 'Terminal'. The menu bar includes 'Arquivo', 'Editar', 'Ver', 'Pesquisar', 'Terminal', and 'Ajuda'. The command prompt shows 'aluno: ~\$./alocacao'. The output of the program is displayed as 'Local = 5' followed by 'Dinamica = 99' on the next line. The terminal has a light gray background and a dark gray title bar.

Execução do código alocaao.c

2.3. Código fatorial.c

Para análise de um exemplo recursivo, foi criado o seguinte código:

```
1 #include <stdio.h>
2 long fatorial(int n) {
3     long parcial;
4     printf("inicio: n = %d\n", n);
5     if (n < 2)
6         parcial = 1;
7     else
8         parcial = n * fatorial(n - 1);
9     printf("final : n = %d, parcial = %ld\n", n, parcial);
10    return parcial;
11 }
12 int main() {
13     printf("Fatorial(4) = %ld\n", fatorial(4));
14     return 0;
15 }
```

Foi realizado a compilação e execução do código:

Figura 12: Elaborada pelo Autor

```
aluno: ~$ gcc fatorial.c -o fatorial
aluno: ~$ ./fatorial
inicio: n = 4
inicio: n = 3
inicio: n = 2
inicio: n = 1
final : n = 1, parcial = 1
final : n = 2, parcial = 2
final : n = 3, parcial = 6
final : n = 4, parcial = 24
Fatorial(4) = 24
aluno: ~$ █
```

Execução do código fatorial.c

3. MMU (Memory Management Unit)

A MMU (Memory Management Unit), tem a função de realizar a tradução dos endereços lógicos, que fica dentro do escopo da visão do processo, para o endereço físico, que se trata da localização real na memória RAM. Quando um programa é compilado e executado, o sistema operacional disponibiliza uma parte da memória RAM para o processo conseguir operar. Toda vez que o processador tentar executar uma função de escrita ou leitura, é enviado um endereço virtual. A MMU consulta sua tabela de paginação armazenada na RAM e traduz o endereço virtual para o endereço físico correspondente na memória RAM.